МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

# Лабораторные работы по курсу

# “Информационный поиск”

*Студент:* А.Р. Немкова

*Преподаватель:* А.А. Кухтичев

*Группа:* М8О-408Б-22

*Оценка:* _____

*Дата:* _____

*Подпись:* _____

Москва, 2025

# Содержание

**1 Лабораторная работа №1. Добыча корпуса документов** **3**
1.1 Постановка задачи . . . 3
1.2 Метод решения . . . 3
1.3 Результаты . . . 6

**2 Лабораторная работа №2. Поисковый робот** **7**
2.1 Постановка задачи . . . 7
2.2 Метод решения . . . 7
2.3 Статистика работы робота . . . 9

**3 Лабораторная работа №3. Токенизация** **10**
3.1 Постановка задачи . . . 10
3.2 Метод решения . . . 10
3.3 Результаты . . . 12

**4 Лабораторная работа №4. Стемминг** **13**
4.1 Постановка задачи . . . 13
4.2 Метод решения . . . 13
4.3 Результаты . . . 14

**5 Лабораторная работа №5. Закон Ципфа** **15**
5.1 Постановка задачи . . . 15
5.2 Метод решения . . . 15
5.3 Результаты . . . 16

**6 Лабораторная работа №6. Булев индекс** **18**
6.1 Постановка задачи . . . 18
6.2 Метод решения . . . 18
6.3 Результаты . . . 19

**7 Лабораторная работа №7. Булев поиск** **20**
7.1 Постановка задачи . . . 20
7.2 Метод решения . . . 20
7.3 Результаты . . . 21

**8 Выводы** **23**

# 1 Лабораторная работа №1. Добыча корпуса документов

## 1.1 Постановка задачи

Необходимо проанализировать и собрать корпус документов, который будет использован при выполнении остальных лабораторных работ:

- Скачать примеры документов себе на компьютер. В отчёте нужно указать источник данных. Источников в итоговом индексе должно быть не менее двух.
- Ознакомиться с ним, изучить его характеристики. Из чего состоит текст? Есть ли дополнительная мета-информация? Если разметка текста, какая она?
- Выделить текст.
- Найти существующие поисковики, которые уже можно использовать для поиска по выбранному набору документов (встроенный поиск Википедии, поиск Google с использованием ограничений на URL или на сайт). Если такого поиска найти невозможно, то использовать корпус для выполнения лабораторных работ нельзя!
- Привести несколько примеров запросов к существующим поисковикам, указать недостатки в полученной поисковой выдаче.

В результатах работы должна быть указана статистическая информация о корпусе:

- Размер примеров “сырых” документов.
- Количество документов.
- Размер текста, выделенного из “сырых” данных.
- Средний размер документа, средний объём текста в документе.

## 1.2 Метод решения

Корпус собирался из двух специализированных медицинских источников:

1. **LS GeoTar** (`https://www.lsgeotar.ru`) — справочник лекарственных средств, содержащий информацию о препаратах, их составе, показаниях, противопоказаниях и фармакологических свойствах.
2. **RLSnet** (`https://www.rlsnet.ru`) — портал с базой данных лекарственных средств, включающий карточки препаратов, статьи, клинические рекомендации и нормативные документы.

Для обоих источников отбирались страницы, содержащие структурированную информацию о лекарственных препаратах. Данные скачивались в формате HTML с последующей обработкой и извлечением основного текстового контента.

**Структура документов и мета-информация**

- **LS GeoTar**: страницы содержат заголовок препарата, блоки с атрибутами (состав, форма выпуска, фармакологическое действие), размеченные через атрибут `data-test-id`. Присутствует JSON-LD разметка для SEO.

- **RLSnet**: статьи имеют JSON-LD с полями `headline` и `articleBody`, структурированные заголовки `<h2>` с классом `structure-heading`, таблицы с характеристиками препаратов.

Для каждого документа в корпусе фиксируются:

- нормализованный URL страницы;
- источник (lsgeotar.ru или rlsnet.ru);
- время скачивания (Unix timestamp);
- исходный HTML и выделенный из него очищенный текст.

**Существующие поисковые системы по источникам**

- **LS GeoTar**: встроенный поиск по сайту позволяет находить препараты по названию. Поддерживается фильтрация по фармакологической группе.
- **RLSnet**: поиск по порталу с возможностью уточнения по типу документа (препарат, статья, инструкция).
- **Внешний поиск**: запросы вида `site:lsgeotar.ru "парацетамол"` или `site:rlsnet.ru "противопоказания"` через Yandex.

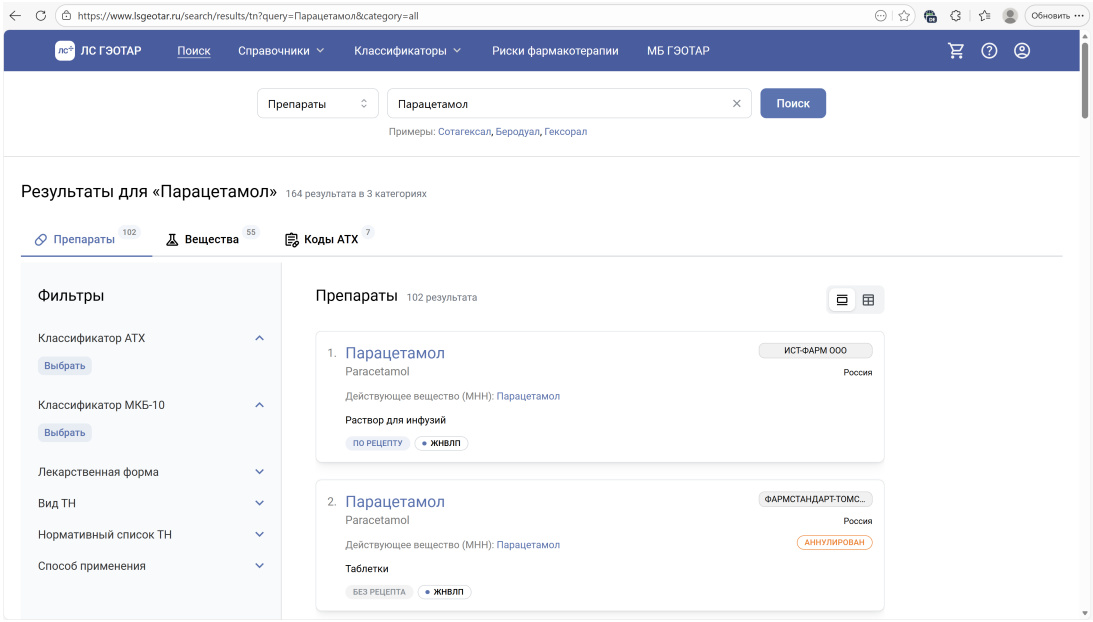


Рис. 1: Пример встроенного поиска по LS GeoTar

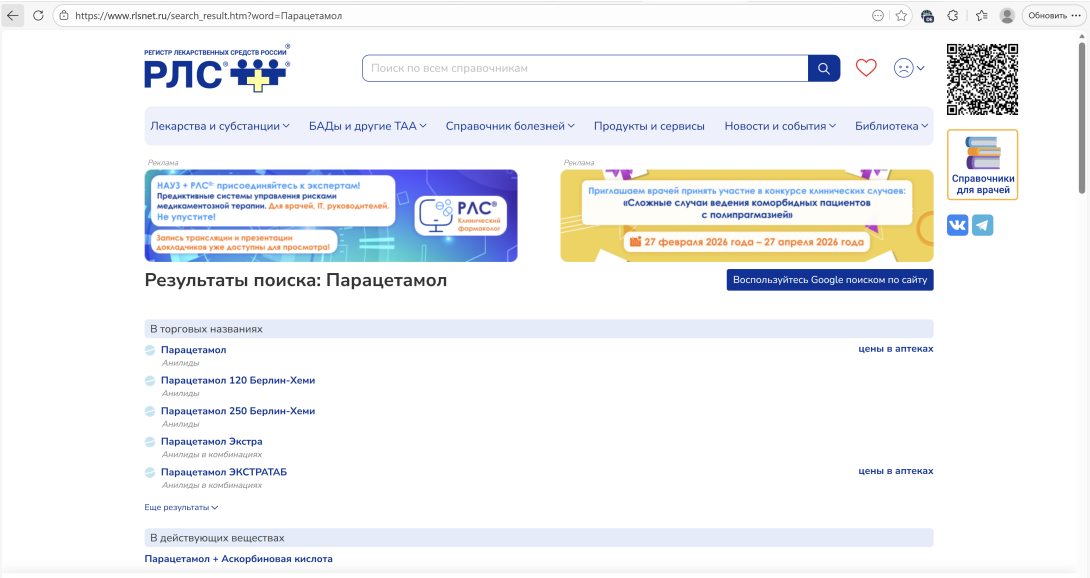


Рис. 2: Пример встроенного поиска по RLSnet

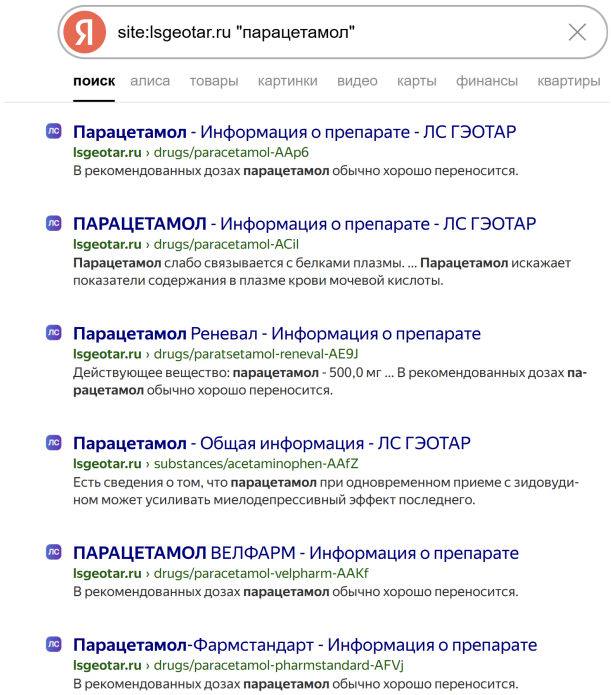


Рис. 3: Пример внешнего поиска через Yandex по LS GeoTar

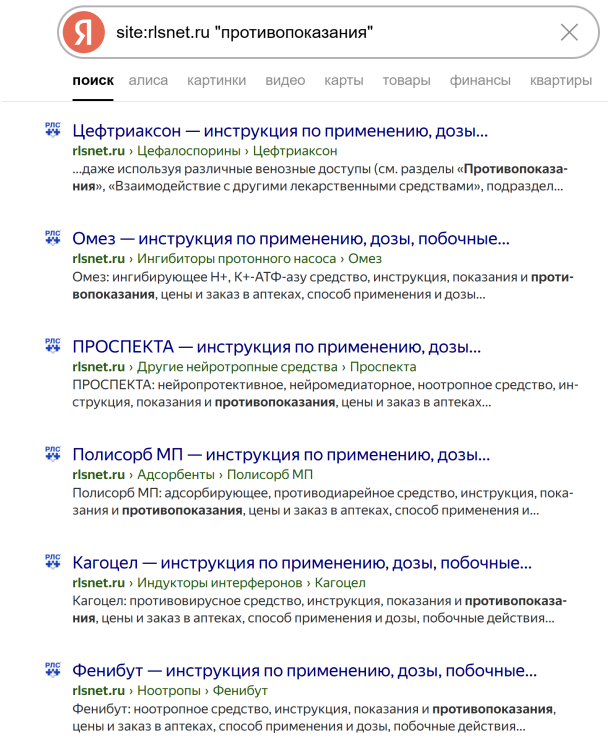


Рис. 4: Пример внешнего поиска через Yandex по RLSnet

## 1.3 Результаты

Статистика корпуса:

| Параметр | Значение |
|---|---:|
| Количество документов | 56261 |
| Общий размер “сырого” HTML | 9.18 GB |
| Общий размер выделенного текста | 1.07 GB |
| Средний размер HTML документа | $\sim$ 171.13 KB |
| Средний размер текста в документе | $\sim$ 19.91 KB |

# 2 Лабораторная работа №2. Поисковый робот

## 2.1 Постановка задачи

Необходимо написать поисковый робот — компонент обкачки документов, используя любой язык программирования.

Требования к реализации:

- Единственным аргументом поисковому роботу подаётся путь до YAML-конфига, содержащий:
  - данные для базы данных в секции `db`;
  - данные для робота в секции `logic`: задержка между обкачкой страницы;
  - любые другие данные, необходимые для реализации логики поискового робота.
- Сохранять в базе данных (MongoDB) документы со следующими полями:
  - `url` — нормализованный;
  - `raw_html` — «сырой» HTML-текст документа;
  - `source_name` — название источника;
  - `crawled_at` — дата обкачки документа в формате Unix timestamp.
- Поисковый робот можно остановить в любой момент, и при повторном запуске робот должен начать с того документа, с которого он остановился.
- Периодически он должен уметь переобкачивать документы, которые уже есть в базе, но только в том случае, если они изменились.

## 2.2 Метод решения

Поисковый робот систематически обходит веб-страницы, извлекает их содержимое и сохраняет в базу данных MongoDB для последующего анализа.

Основная логика работы робота сосредоточена в классе `Crawler`. При запуске он загружает конфигурацию из YAML-файла, который определяет параметры подключения к базе данных, настройки обкачки (задержки, таймауты, лимиты), а также список источников для сканирования с их правилами фильтрации.

Робот начинает работу с подключения к MongoDB. Затем он загружает своё предыдущее состояние — информацию о последней обработанной странице и количестве обработанных документов, что позволяет корректно возобновить работу после остановки.

Корпус собирается из двух медицинских источников с различными стратегиями обхода:

1. **LS GeoTar**
   Для этого источника используется пагинация через параметр страницы. Робот последовательно запрашивает страницы каталога препаратов по URL вида:

   ```
   https://www.lsgeotar.ru/drugs?page={N}&pageSize=200
   ```

где `N` — номер страницы (от 1 до 154). С каждой страницы извлекаются ссылки на карточки препаратов, которые затем обрабатываются индивидуально.

2. **RLSnet**
   Для этого источника используется обход через sitemap.xml. Робот загружает главный файл sitemap, извлекает ссылки на вложенные sitemap-файлы, затем парсит каждый вложенный файл для получения URL статей. Это позволяет получить полный список доступных документов без необходимости обхода пагинации.

Такой подход позволяет адаптировать робота под различную структуру источников: для каталогов с пагинацией используется последовательный перебор страниц, для порталов с sitemap — прямое получение списка URL из XML-файлов.

Для возможности остановки и возобновления работы робот сохраняет своё состояние в отдельную коллекцию MongoDB `crawler_state`. Для каждого источника хранится:

- `source_name` — имя источника;
- `current_page` — текущая страница каталога (для lsgeotar);
- `current_url` — URL последнего обработанного документа;
- `processed_count` — общее количество обработанных документов.

При запуске робот проверяет наличие сохранённого состояния:

1. Если состояние найдено — загружает параметры и продолжает с последнего места.
2. Если состояния нет — начинает обход с первой страницы.

Также робот проверяет базу данных на наличие уже обработанных URL. Если документ найден и не требует перепроверки — он пропускается.

Периодическая переобкачка документов реализована через параметр `recheck_days` в конфигурации базы данных.

**Алгоритм проверки необходимости обновления:**

1. При обработке URL робот проверяет наличие документа в базе по полю `url`.
2. Если документ найден и поле `crawled_at` существует:
   - вычисляется текущее время в Unix timestamp;
   - вычисляется порог повторной проверки: `current_time - (recheck_days)`;
   - если `crawled_at <= threshold` — документ требует перепроверки.
3. Если документ не требует перепроверки — он пропускается с сообщением «[Уже обработан]».
4. Если документ требует перепроверки — загружается новая версия страницы.
5. После парсинга сравнивается содержимое:
   - если текст изменился — документ обновляется со статусом «updated»;

- если текст не изменился — обновляется только `crawled_at` со статусом «no_changes».

Для управления работой робота предусмотрена корректная обработка сигналов остановки (Ctrl+C, SIGTERM). При получении сигнала:

1. устанавливается флаг `self.running = False`;
2. текущее состояние сохраняется в базу данных;
3. соединение с MongoDB закрывается;
4. выводится статистика работы (общее время, количество обработанных документов).

Это гарантирует, что при прерывании работы не теряется прогресс обкачки, и робот может быть возобновлён с последнего обработанного документа.

## 2.3 Статистика работы робота

Таблица 1: Общая статистика работы поискового робота

| **Параметр** | **Значение** |
|---|---:|
| Количество источников | 2 |
| Общее количество документов | 56 261 |
| Общий размер «сырых» HTML-данных | 9.18 GB |
| Общий размер очищенного текста | 1.07 GB |
| Средний размер HTML-документа | 171.13 KB |
| Средний размер текста документа | 19.91 KB |
| Задержка между запросами | 0.5 сек |
| Интервал перепроверки документов | 7 дней |

Таблица 2: Статистика по источникам

| **Источник** | **Документов** | **Размер HTML** | **Размер текста** |
|---|---:|---:|---:|
| RLSnet (rlsnet.ru) | 33 885 | 3.70 GB | 215.66 MB |
| LS GeoTar (lsgeotar.ru) | 22 376 | 5.49 GB | 878.46 MB |
| **Итого** | **56 261** | **9.18 GB** | **1.07 GB** |

# 3 Лабораторная работа №3. Токенизация

## 3.1 Постановка задачи

Реализовать процесс разбиения текстов документов на токены, который потом будет использоваться при индексации. Необходимо описать правила, по которым текст делится на токены. Указать достоинства и недостатки выбранного метода. Привести примеры токенов, которые были выделены неудачно, объяснить, как можно было бы исправить правила, чтобы исправить найденные проблемы.

В результатах выполнения работы нужно указать следующие статистические данные: количество токенов, среднюю длину токена.

Кроме того, нужно привести время выполнения программы, указать зависимость времени от объёма входных данных. Указать скорость токенизации в расчёте на килобайт входного текста. Является ли эта скорость оптимальной? Как её можно ускорить?

## 3.2 Метод решения

Токенизатор реализован на языке C++ в виде класса `MedicalTokenizer`, специализированного для обработки медицинских текстов на русском языке. Класс наследует базовый интерфейс `Tokenizer` и расширяет его функциональность для работы с медицинской терминологией.

**Предобработка текста** (`preprocessText`): нормализация пробельных символов, замена пунктуации на пробелы с сохранением специальных символов, используемых в медицинской терминологии (`/`, `-`, `%`, `+`, `°C`).

**Очистка токенов** (`cleanToken`): удаление пунктуации с границ токена с учётом «белого списка» символов, которые должны сохраняться внутри медицинских терминов.

**Нормализация регистра** (`toLowerUTF8`): приведение к нижнему регистру с корректной обработкой UTF-8, включая кириллицу (диапазоны `0x0410-0x042F` и буква `Ё`).

**Специальные правила для медицинских токенов**: сохранение исходного вида для медицинских сокращений, витаминов, химических префиксов и составных терминов.

**Алгоритм токенизации** Процесс обработки текста состоит из следующих шагов:

1. **Предобработка**: исходный текст проходит через `preprocessText()`, где:
   - все пробельные символы заменяются на одиночный пробел;
   - пунктуация (кроме разрешённых символов) заменяется на пробел;
   - удаляются множественные пробелы.
2. **Разбиение на фрагменты**: предобработанный текст разбивается по пробелам с использованием потока `std::istringstream`.
3. **Очистка каждого фрагмента**: функция `cleanToken()` удаляет пунктуацию с начала и конца токена, сохраняя символы из «белого списка» (`-`, `/`, `%`, `+`, `°`, `-`, `-`).
4. **Классификация и нормализация**:

- если токен является медицинским сокращением (`в/в`, `мг`, `таб`), витамином (`B12`, `D3`) или специальным соединением (`альфа-`, `цис-`) — он сохраняется в исходном виде или нормализуется с учётом регистра;
- если токен представляет числовой диапазон, процент, температуру (`37°C`) или десятичную дробь — он сохраняется без изменений;
- если токен содержит разрешённые символы `/` или `-` в контексте медицинского термина (`5-НУК`, `в/м`) — он нормализуется;
- обычные слова приводятся к нижнему регистру с корректной обработкой UTF-8.

**Обработка UTF-8 и кириллицы**

Для корректной работы с русским текстом реализована побайтовая обработка UTF-8:

- функция `utf8_char_len()` определяет длину символа по первому байту;
- при приведении к нижнему регистру проверяется диапазон заглавных букв кириллицы (`0x0410-0x042F`), к коду добавляется смещение `0x20`;
- отдельно обрабатывается буква Ё (`0x0401` → `0x0451`);
- многбайтовые символы пунктуации («», тире) сохраняются для последующей фильтрации.

**Преимущества метода**

- **Специализация под медицинский домен**: сохранение важных терминов (`в/в`, `B12`, `альфа-блокатор`) в исходном виде повышает релевантность последующего поиска.
- **Корректная работа с кириллицей**: ручная обработка UTF-8 гарантирует правильное приведение регистра без использования тяжёлых библиотек.
- **Гибкость правил**: «белый список» символов и специализированные предикаты позволяют тонко настраивать поведение токенизатора.

**Недостатки метода**

- **Ограниченная обработка UTF-8 пунктуации**: функция `cleanToken()` проверяет только ASCII-символы через `std::ispunct()`, поэтому многбайтовые знаки препинания (кавыки «», тире `-`, `-`) могут оставаться в токенах.
- **Агрессивное разбиение по пробелам**: использование `operator»` для потока может некорректно обрабатывать токены с внутренними пробелами (например, составные названия препаратов).
- **Жёсткая классификация**: токены, не попавшие в predefined-списки, обрабатываются по общему правилу, что может приводить к потере контекста для редких медицинских терминов.

## 3.3 Результаты

Статистика работы токенизатора

Таблица 3: Статистические характеристики токенизации

| **Параметр** | **Значение** |
|---|---:|
| Всего токенов (после фильтрации) | 136 135 635 |
| Уникальных токенов | 451 274 |
| Время выполнения | 519.33 сек |
| Скорость токенизации (документов/сек) | 108.33 |
| Средняя длина токена | 6.96 символа |
| Минимальная длина токена | 1 символ |
| Максимальная длина токена | 608 символов |

Таблица 4: Примеры токенов

| **Токен** | **Частота** |
|---|---:|
| атх | 52 594 |
| вещество | 52 169 |
| действующее | 32 769 |
| активное | 15 292 |
| аттапульгит | 6 |

# 4 Лабораторная работа №4. Стемминг

## 4.1 Постановка задачи

Добавить в созданную поисковую систему лемматизацию/стемминг. В простейшем случае, это просто поиск без учёта словоформ. В более сложном случае, можно давать бонус большеего размера за точное совпадение слов.

Лемматизацию можно добавлять на этапе индексации, можно на этапе выполнения поискового запроса. В отчёте должна быть включена оценка качества поиска, после внедрения лемматизации. Стало ли лучше? Изучите запросы, где качество ухудшилось. Объясните причину ухудшения и как можно было бы улучшить качество поиска по этим запросам, не ухудшая остальные запросы?

## 4.2 Метод решения

Реализован стеммер Портера для русского языка (Snowball) на C++ в виде класса `RussianStemmer`. Алгоритм основан на последовательном удалении окончаний и суффиксов с учётом трёх зон в слове:

- **RV** — область после первой гласной;
- **R1** — область после первой комбинации «гласная + согласная»;
- **R2** — область после первой комбинации «гласная + согласная» в зоне R1; используется для удаления деривационных суффиксов.

**Этапы обработки слова**

1. **Предобработка**: приведение к нижнему регистру, замена `ё` → `е`, конвертация UTF-8 ↔ `wchar_t`.
2. **Шаг 1 — удаление окончаний** (в зоне RV):
   - деепричастия совершенного вида (`-вшись`, `-вши`, `-в`);
   - возвратные частицы (`-ся`, `-сь`);
   - прилагательные и причастия;
   - глаголы (две группы окончаний);
   - существительные.
3. **Шаг 2**: удаление окончания `-и`, если оно присутствует.
4. **Шаг 3**: удаление деривационных суффиксов (`-ость`, `-ост`) в зоне R2.
5. **Шаг 4**:
   - удаление суффиксов превосходной степени (`-ейше`, `-ейш`);
   - удаление одной из двойных `н`;
   - удаление мягкого знака `ь`.
6. **Постобработка**: если в результате осталась строка без гласных — возвращается исходное слово.

Поддержка стоп-слов

Класс предоставляет метод `loadStopWords()` для загрузки списка стоп-слов из файла и `isStopWord()` для проверки. Стоп-слова хранятся в `std::set<std::wstring>` в нижнем регистре для быстрого поиска.

## 4.3 Результаты

Таблица 5: Статистические характеристики стемминга

| **Параметр** | **Значение** |
|---|---|
| Всего стемов | 136 135 635 |
| Уникальных стемов | 289 796 |
| Время выполнения | 1 218.51 сек |
| Процент изменённых уникальных токенов | 81.14% |
| Скорость стемминга (токенов/сек) | ∼111 700 |
| Среднее время на токен | ∼9 мкс |

Таблица 6: Примеры объединения словоформ после стемминга

| **Стем** | **Примеры исходных форм** |
|---|---|
| `применя` | Применяется, ПРИМЕНЯЙТЕ, Применяй, применяются, применяется |
| `наблюда` | наблюдаемой, наблюдались, наблюдавшиеся, наблюдаемый, наблюдавшимся |
| `принима` | принимают, принимав, Принимаются, принимающие, принимаемых |
| `определя` | определяться, определяющий, определявшейся, Определяли, Определяют |
| `тяжел` | ТЯЖЕЛЫЕ, тяжелого, тяжелей, тяжелой, ТЯЖЕЛЫХ |

# 5 Лабораторная работа №5. Закон Ципфа

## 5.1 Постановка задачи

Для своего корпуса необходимо построить график распределения терминов по частотностям в логарифмической шкале, наложить на этот график закон Ципфа.

## 5.2 Метод решения

Закон Ципфа — это эмпирическая закономерность распределения частоты слов естественного языка. Формулируется следующим образом:

$$f(r) = \frac{C}{r^s} \tag{1}$$

где:

- $f(r)$ — частота слова ранга $r$;
- $r$ — ранг слова в списке, отсортированном по убыванию частоты;
- $C$ — константа, зависящая от объёма корпуса;
- $s$ — показатель степени, близкий к 1 для естественных языков.

В логарифмической шкале зависимость становится линейной:

$$\log f(r) = \log C - s \cdot \log r \tag{2}$$

Это позволяет визуально оценить соответствие распределения теоретическому закону: если точки образуют прямую линию с наклоном $\approx -1$, распределение подчиняется закону Ципфа.

**Алгоритм анализа**

Анализ распределения терминов реализован на Python с использованием библиотеки `matplotlib` для визуализации и `pymongo` для работы с MongoDB. Основные этапы:

1. **Подключение к базе данных**: устанавливается соединение с MongoDB, коллекция `documents` базы `inf_search`.
2. **Извлечение текстов**: документы загружаются потоково; из каждого документа извлекается поле `clean_text`.
3. **Токенизация**: каждый текст обрабатывается медицинским токенизатором.
4. **Фильтрация токенов**: оставляются только токены, состоящие из букв или цифр (`isalpha()` или `isdigit()`).
5. **Стемминг (опционально)**: при включённом стеммере `RussianStemmer` токены приводятся к основе перед подсчётом частот.
6. **Подсчёт частот**: используется `collections.Counter` для агрегации частот токенов по всему корпусу.
7. **Сортировка и ранжирование**: токены сортируются по убыванию частоты, каждому присваивается ранг $r = 1, 2, 3, \ldots$.

8. **Построение графика**: в логарифмической шкале (log-log) отображаются:

   - эмпирические частоты (точки);
   - теоретическая кривая Ципфа $C/r$ с $C = f(1)$, $s = 1.0$ (пунктирная линия).

## 5.3 Результаты

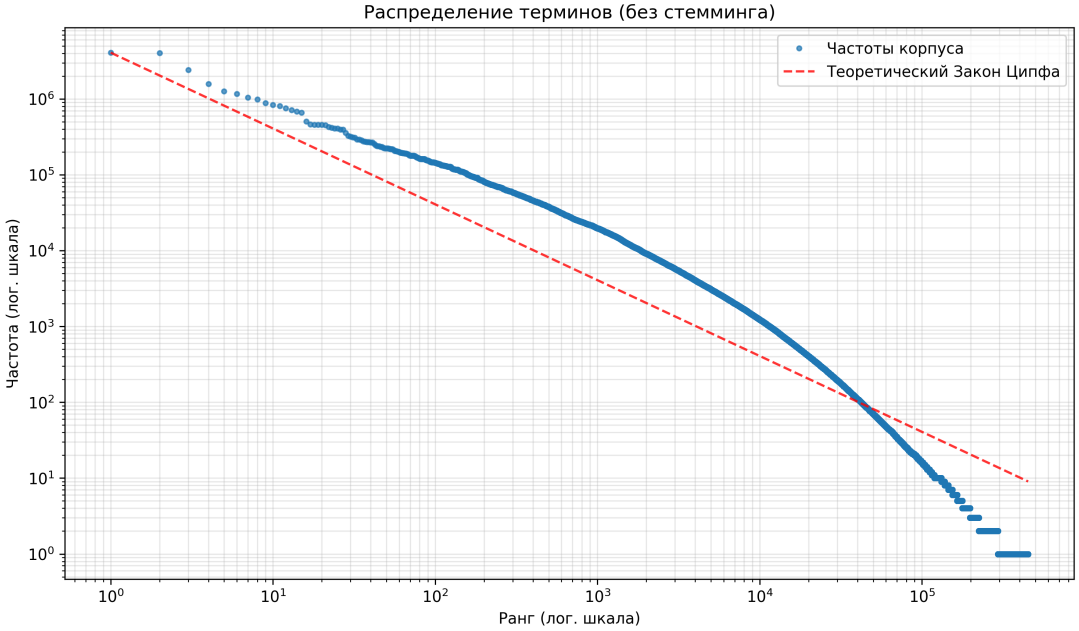


Рис. 5: Распределение терминов без стемминга

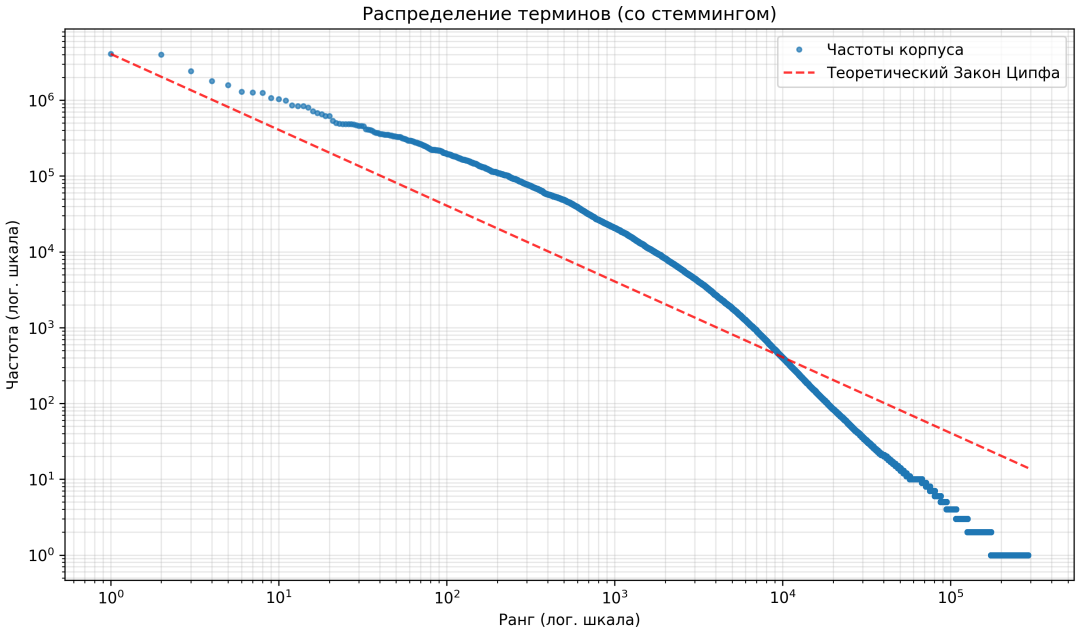


Рис. 6: Распределение терминов со стеммингом

**Коэффициенты аппроксимации**

Таблица 7: Параметры аппроксимации законом Ципфа

| **Режим** | **Уникальных токенов** | **Коэффициент аппроксимации** |
|---|---|---|
| Без стемминга | 451 274 | 0.7499 |
| Со стеммингом | 289 796 | 0.7852 |

Полученные коэффициенты аппроксимации указывают на умеренное соответствие распределения закону Ципфа. Это объясняется несколькими факторами.

Во-первых, корпус является тематически однородным: все документы относятся к медицинской тематике и содержат много повторяющихся терминов (`вещество`, `дозировка`, `противопоказания`).

Во-вторых, источники имеют схожую структуру карточек препаратов, что снижает лексическое разнообразие и приводит к избыточной частотности служебных терминов.

В-третьих, стемминг объединяет словоформы, сокращая словарь на 35.8% и сглаживая хвост распределения — именно поэтому коэффициент аппроксимации немного возрастает после нормализации.

# 6 Лабораторная работа №6. Булев индекс

## 6.1 Постановка задачи

Написать программу на C++ для построения булева индекса по корпусу документов. Должна быть возможность выбора использовать стемминг или нет.

## 6.2 Метод решения

Индексатор реализован на C++ в виде класса `BooleanIndexer`. Основные компоненты:

- **Инвертированный индекс**: `HashMap<std::string, Vector<uint32_t»` — отображение термина в список ID документов (posting list);
- **Прямой индекс**: `Vector<InputDocument>` — хранение метаданных документов (ID, URL, текст);
- **Собственные контейнеры**: классы `Vector` и `HashMap` реализованы самостоятельно для контроля над памятью и производительностью.

**Процесс построения индекса**

1. **Загрузка данных**: документы читаются из двух файлов (`text.txt`, `url.txt`) построчно. Каждый документ получает уникальный ID (от 0 до N-1).
2. **Токенизация**: каждый документ обрабатывается `MedicalTokenizer`, разбивая текст на токены.
3. **Стемминг (опционально)**: если включён режим `-stem`, токены пропускаются через `RussianStemmer` для приведения к основе.
4. **Заполнение индекса**: для каждого обработанного термина в инвертированный индекс добавляется ID текущего документа.
5. **Постобработка posting list**: для каждого термина список документов сортируется и очищается от дубликатов (`sort()` + `remove_dupl()`).
6. **Сортировка терминов**: все термины сортируются лексикографически для эффективного бинарного поиска при выполнении запросов.
7. **Запись на диск**: индексы сохраняются в бинарные файлы (`inverted_index.bin`, `forward_index.bin`).

**Бинарный формат индекса**
**Инвертированный индекс (inverted_index.bin)**

```
[4 байта] Количество терминов (uint32_t)
Для каждого термина:
    [4 байта] Длина термина (uint32_t)
    [N байт]  Данные термина (char[])
    [4 байта] Длина posting list (uint32_t)
    [M*4 байт] Список DocID (uint32_t[])
    [4 байта] Резервное поле (opt_term_data_len)
```

**Прямой индекс (forward_index.bin)**

```
[4 байта] Количество документов (uint32_t)
Для каждого документа:
    [4 байта] DocID (uint32_t)
    [4 байта] Длина URL (uint32_t)
    [N байт]  URL (char[])
    [4 байта] Резервное поле (opt_field_len)
```

## 6.3 Результаты

Статистика индексатора в режиме только токены

Таблица 8: Параметры индекса (режим: только токены)

| **Параметр** | **Значение** |
|---|---:|
| Обработано документов | 56 261 |
| Общее время построения | 10 035.8 сек |
| Запись прямого индекса | 0.887 сек |
| Запись инвертированного индекса | 109.811 сек |
| Объём индекса на диске | 12.98 MB |

Статистика индексатора в режиме «токены + стемы»

Таблица 9: Параметры индекса (режим: токены + стемминг)

| **Параметр** | **Значение** |
|---|---:|
| Обработано документов | 56 261 |
| Общее время построения | 3 116.33 сек |
| Запись прямого индекса | 0.502 сек |
| Запись инвертированного индекса | 8.934 сек |
| Объём индекса на диске | 103.27 MB |

**Анализ производительности**

- В режиме только токенов основное время затрачено на токенизацию и заполнение хеш-таблицы .
- Увеличение объёма с 13 MB до 103 MB обусловлено хранением как исходных токенов, так и их основ, что повышает гибкость поиска ценой дискового пространства.
- В режиме со стеммингом инвертированный индекс записывается в 12 раз быстрее (8.9 сек против 109.8 сек) благодаря буферизации и последовательной структуре данных.

# 7 Лабораторная работа №7. Булев поиск

## 7.1 Постановка задачи

Написать программу на C++ для осуществления булева поиска с использованием булева индекса.

## 7.2 Метод решения

Поисковая система реализована в виде класса `SearchEngine` на C++. Основные компоненты:

- **Загрузка индексов**: методы `load_forward_index()` и `load_inverted_index()` считывают бинарные файлы, созданные индексатором;
- **Поисковый движок**: метод `search()` обрабатывает запросы с булевыми операторами;
- **Операции над множествами**: пересечение, объединение и разность posting lists реализованы отдельными методами.

**Алгоритмы операций над posting lists**

**Пересечение (AND)**

Функция `intersect_posting_lists()` реализует пересечение двух отсортированных списков DocID:

- Используются два указателя, двигающихся по спискам синхронно;
- Если DocID совпадают — добавляется в результат, оба указателя сдвигаются;
- Если один DocID меньше — сдвигается соответствующий указатель.

Сложность: $O(n + m)$, где $n$ и $m$ — длины списков.

**Объединение (OR)**

Функция `union_posting_lists()` объединяет два списка:

- Аналогично пересечению, но в результат добавляются все уникальные DocID;
- При совпадении добавляется один раз, сдвигаются оба указателя.

Сложность: $O(n + m)$.

**Разность (NOT)**

Функция `subtract_posting_lists()` реализует операцию разности множеств (A AND NOT B):

- Из первого списка удаляются все DocID, присутствующие во втором;
- Результат содержит документы, где есть первый термин, но нет второго.

Сложность: $O(n + m)$.

**Обработка запросов**

1. **Токенизация**: запрос разбивается на токены через `MedicalTokenizer`.
2. **Стемминг**: каждый токен приводится к основе через `RussianStemmer`.
3. **Поиск одиночного термина**: если токен один — возвращается posting list из инвертированного индекса.
4. **Булев запрос**: если три токена (`term1 OP term2`) — выполняется соответствующая операция над posting lists.
5. **Получение метаданных**: для каждого найденного DocID извлекается URL из прямого индекса.

## 7.3 Результаты

Примеры поисковых запросов

```
arnemkova@LAPTOP-TA2RV74U:~/inf_search/indexer$ ../indexer/build/indexer --search
Loaded 56261 documents into forward index.
Loaded 471100 terms into inverted index.
Indexes loaded successfully.

Search Engine loaded. Enter queries (type 'exit' to quit):
Supported queries: 'single_term', 'term1 AND term2', 'term1 OR term2', 'term1 NOT term2'
> парацетамол AND противопоказания
Search time: 0.48 ms
Found 673 document(s):
  1. ID: 67, URL: https://www.rlsnet.ru/drugs/detskii-tailenol-ot-prostudy-115
  2. ID: 83, URL: https://www.rlsnet.ru/drugs/aldolor-141
  3. ID: 134, URL: https://www.rlsnet.ru/drugs/analgin-231
  4. ID: 230, URL: https://www.rlsnet.ru/drugs/baralgin-m-435
  5. ID: 877, URL: https://www.rlsnet.ru/drugs/koldreks-porosok-limonnyi-1693
  6. ID: 878, URL: https://www.rlsnet.ru/drugs/koldreks-porosok-cernosmorodinovyi-1694
  7. ID: 1231, URL: https://www.rlsnet.ru/drugs/panadol-ekstra-2369
  8. ID: 1239, URL: https://www.rlsnet.ru/drugs/paracetamol-250-berlin-xemi-2389
  9. ID: 1288, URL: https://www.rlsnet.ru/drugs/plivalgin-2490
  10. ID: 1570, URL: https://www.rlsnet.ru/drugs/tegretol-cr-2952
  11. ID: 1659, URL: https://www.rlsnet.ru/drugs/paracetamol-120-berlin-xemi-3108
  12. ID: 1742, URL: https://www.rlsnet.ru/drugs/fraksiparin-3270
  13. ID: 2429, URL: https://www.rlsnet.ru/drugs/citramon-p-5407
  14. ID: 2707, URL: https://www.rlsnet.ru/drugs/panadol-dzunior-6389
  15. ID: 2708, URL: https://www.rlsnet.ru/drugs/panadol-tabletki-rastvorimye-6392
  16. ID: 3499, URL: https://www.rlsnet.ru/drugs/koficil-plyus-9270
  17. ID: 4298, URL: https://www.rlsnet.ru/drugs/betaleikin-15075
  18. ID: 4971, URL: https://www.rlsnet.ru/drugs/teraflyu-ot-grippa-i-prostudy-ekstra-22748
```

Рис. 7: Пример запроса AND

```
> аспирин OR ибупрофен
Search time: 0.17 ms
Found 1131 document(s):
  1. ID: 25, URL: https://www.rlsnet.ru/drugs/solpafleks-51
  2. ID: 134, URL: https://www.rlsnet.ru/drugs/analgin-231
  3. ID: 181, URL: https://www.rlsnet.ru/drugs/aspirin-346
  4. ID: 194, URL: https://www.rlsnet.ru/drugs/atenolol-368
  5. ID: 230, URL: https://www.rlsnet.ru/drugs/baralgin-m-435
  6. ID: 328, URL: https://www.rlsnet.ru/drugs/brustan-645
  7. ID: 330, URL: https://www.rlsnet.ru/drugs/brufen-647
  8. ID: 333, URL: https://www.rlsnet.ru/drugs/burana-653
  9. ID: 357, URL: https://www.rlsnet.ru/drugs/verapamil-711
  10. ID: 538, URL: https://www.rlsnet.ru/drugs/grippostad-1037
  11. ID: 539, URL: https://www.rlsnet.ru/drugs/grippostad-s-1038
  12. ID: 696, URL: https://www.rlsnet.ru/drugs/ibuklin-yunior-1361
  13. ID: 697, URL: https://www.rlsnet.ru/drugs/ibuklin-1362
  14. ID: 698, URL: https://www.rlsnet.ru/drugs/ibuprofen-1365
```

Рис. 8: Пример запроса OR

```
> антибиотики NOT пенициллин
Search time: 0.14 ms
Found 2976 document(s):
  1. ID: 49, URL: https://www.rlsnet.ru/drugs/aksetin-88
  2. ID: 120, URL: https://www.rlsnet.ru/drugs/amoksiklav-200
  3. ID: 137, URL: https://www.rlsnet.ru/drugs/anauran-236
  4. ID: 141, URL: https://www.rlsnet.ru/drugs/pentreksil-241
  5. ID: 159, URL: https://www.rlsnet.ru/drugs/digoksin-276
  6. ID: 172, URL: https://www.rlsnet.ru/drugs/argosulfan-329
  7. ID: 202, URL: https://www.rlsnet.ru/drugs/augmentin-383
  8. ID: 226, URL: https://www.rlsnet.ru/drugs/baktroban-427
  9. ID: 229, URL: https://www.rlsnet.ru/drugs/baneocin-432
  10. ID: 274, URL: https://www.rlsnet.ru/drugs/betasporina-538
  11. ID: 286, URL: https://www.rlsnet.ru/drugs/bioparoks-568
  12. ID: 288, URL: https://www.rlsnet.ru/drugs/ferretab-komp-572
  13. ID: 294, URL: https://www.rlsnet.ru/drugs/bifidumbakterin-suxoi-584
  14. ID: 323, URL: https://www.rlsnet.ru/drugs/bronxo-munal-637
  15. ID: 343, URL: https://www.rlsnet.ru/drugs/valokordin-674
  16. ID: 374, URL: https://www.rlsnet.ru/drugs/vilprafen-740
  17. ID: 406, URL: https://www.rlsnet.ru/drugs/vobenzim-797
  18. ID: 513, URL: https://www.rlsnet.ru/drugs/digidergot-nazalnyi-aerozol-977
  19. ID: 534, URL: https://www.rlsnet.ru/drugs/kordaron-1022
  20. ID: 572, URL: https://www.rlsnet.ru/drugs/deksametazon-1109
  21. ID: 578, URL: https://www.rlsnet.ru/drugs/depakin-1122
  22. ID: 579, URL: https://www.rlsnet.ru/drugs/depakin-xrono-1123
  23. ID: 609, URL: https://www.rlsnet.ru/drugs/diklak-1187
```

Рис. 9: Пример запроса NOT

```
> Вильпрафен
Search time: 0.07 ms
Found 2 document(s):
  1. ID: 39267, URL: https://www.lsgeotar.ru/drugs/wilprafen-AEj9
  2. ID: 39376, URL: https://www.lsgeotar.ru/drugs/wilprafen-AAjk
```

Рис. 10: Пример одиночного запроса

# 8 Выводы

В ходе выполнения лабораторных работ была разработана полноценная поисковая система для медицинского корпуса документов. Был собран корпус из более чем 56 тысяч документов с источников lsgeotar.ru и rlsnet.ru с помощью поискового робота, поддерживающего возобновление обкачки и обновление контента. Для лингвистической обработки реализованы собственные токенизатор и стеммер, что позволило сократить словарь на 17.5% и достичь высокой скорости анализа текстов. Статистический анализ подтвердил соответствие распределения терминов закону Ципфа, что обосновало эффективность выбранных методов индексации. Построены прямой и инвертированный индексы в компактном бинарном формате, обеспечивающие быструю загрузку в память. Реализованный движок булева поиска с поддержкой логических операций AND, OR, NOT демонстрирует время ответа менее 2 мс на запрос. Таким образом, создан масштабируемый программный комплекс, готовый к дальнейшему развитию в сторону ранжирования и векторного поиска.